

CONFIDENTIAL ENGINEERING TASK SPECIFICATION

VaultX PoC

Further Development Task List

Organized Trial.pdf-style execution brief with improved spacing, clean module tables, and structured required-work sections.

Project	VaultX Protocol - PoC Further Development
Document Type	Internal Engineering Task Brief
Scope	Frontend-only UX, standalone status/document APIs, and independent Solidity support contracts
Classification	CONFIDENTIAL - Internal Use Only
Date Issued	May 24, 2026

Task Inventory

Ref.	Domain	Deliverable	Timebox	Dependency
FE-11	FRONTEND	Add Investor-Focused Asset Detail Drawer	1 Business Day	Independent from other Group 1 tasks
BE-12	EXPRESS	Add Public VaultX Status API	1 Business Day	Independent from other Group 1 tasks
SC-13	SOLIDITY	Add VaultXTreasuryLedger Contract	1 Business Day	Independent from other Group 1 tasks
FE-21	FRONTEND	Build VaultX Compliance Center Page	3 Business Days	Independent 3-day scope
BE-22	EXPRESS	Add Asset Document Metadata API	3 Business Days	Independent 3-day scope
SC-23	SOLIDITY	Add VaultXDocumentAnchor Contract	3 Business Days	Independent 3-day scope

Table 1. Task inventory with clarified dependency language.

1.0 Execution Context

This Trial-style PDF organizes the uploaded VaultX PoC task list into a senior engineering execution brief. The content has been compacted into structured sections so the Required Work lists are readable and the page space is used more efficiently.

The two groups are intentionally independent. Task Group 1 contains three separate one-business-day tasks. Task Group 2 contains three separate three-business-day tasks. Each task can be assigned to a different frontend, backend, or Solidity developer without relying on another task in the same group.

Delivery Control Matrix

Group	Timebox	Engineering Intent	Exit Standard
Task Group 1	1 Business Day	Add investor-focused asset detail UX, a public backend status API, and an independent treasury ledger contract.	Each task produces a small standalone improvement without backend/Solidity/frontend cross-dependency.
Task Group 2	3 Business Days	Add a standalone compliance center page, document metadata APIs, and an on-chain document anchor contract.	Compliance and disclosure readiness are represented clearly while preserving independent workstream execution.

Table 2. Delivery control matrix.

Formatting Standard Applied

- Required Work sections are grouped into work-area tables instead of unstructured flat bullet lists.
- File paths are kept in compact review-purpose tables to prevent wasted space.
- Expected deliverables remain checklist-style for development review.
- No dependency wording implies that Group 2 requires Group 1 unless the task text explicitly says so.

2.0 Task Group 1 - 1 Business Day

Delivery focus: Standalone one-day improvements: asset detail drawer, public status API, and treasury ledger contract.

FE-11 - Add Investor-Focused Asset Detail Drawer

Domain	Timebox	Module Ref.	Implementation Mode
FRONTEND	1 Business Day	FE-11	Standalone / non-blocking

Technical Target

Add a standalone asset detail drawer to the VaultX frontend so users can review richer RWA asset information from asset cards without depending on backend or Solidity changes.

Current PoC Gap

The current frontend has visual RWA asset cards, but the user cannot open a detailed investor-style view for each asset. Asset cards still feel like marketing tiles instead of structured RWA investment records.

Required Work

Work Area	Organized Required Work
Create a new frontend component	- src/components/assets/AssetDetailDrawer.jsx
Create supporting components	- src/components/assets/AssetMetricGrid.jsx - src/components/assets/AssetRiskSummary.jsx - src/components/assets/AssetActionPanel.jsx - AssetDetailDrawer must open when clicking an asset card. - Use existing frontend asset data only. - Do not require new backend API. - Do not require new Solidity contract.
Displayed investor data (1)	- asset ID - asset name - asset type - location - target APY - occupancy - funding progress
Displayed investor data (2)	- minimum participation amount - risk level - short asset memo - investment status
Panel actions / messaging	- View details - Request access - Go to presale
If wallet is not connected	- keep investment buttons disabled - show Connect Wallet required message

Table. Organized required-work structure for implementation review.

Main Files

Primary file / path	Review purpose
src/components/assets/AssetDetailDrawer.jsx	React page/component surface
src/components/assets/AssetMetricGrid.jsx	React page/component surface
src/components/assets/AssetRiskSummary.jsx	React page/component surface
src/components/assets/AssetActionPanel.jsx	React page/component surface
src/components/gallery/GalleryItems.jsx	React page/component surface
src/components/home/HeroSection.jsx	React page/component surface

Expected Deliverable

- [] User can click an RWA asset card.
- [] Asset detail drawer opens.
- [] Drawer uses current frontend asset data.
- [] Asset information is clearer and more investor-focused.
- [] No backend or Solidity dependency is required.

BE-12 - Add Public VaultX Status API

Domain	Timebox	Module Ref.	Implementation Mode
EXPRESS	1 Business Day	BE-12	Standalone / non-blocking

Technical Target

Create a simple standalone Express API endpoint that returns public VaultX PoC status information for frontend or external monitoring use.

Current PoC Gap

The project does not have a simple backend status endpoint that describes the PoC service state, public VaultX address, current environment, and visible module readiness.

Required Work

Work Area	Organized Required Work
Create or update backend route	- GET /api/status
API response contract (1)	- { - "success": true, - "data": { - "service": "vaultx-poc-api", - "environment": "development", - "publicVaultAddress": "0xBb569C738f56348B21a84D520f679fe41Fd01cc5", - "frontendReady": true, - }
API response contract (2)	- "contractDeploymentReady": false, - "assetRegistryReady": false, - "lastUpdated": "ISO_DATE_STRING" - } - }
Add basic uptime value	- uptimeSeconds
Add static version field	- version - This API must not call blockchain RPC. - This API must not depend on frontend. - This API must not depend on asset data routes. - This API must work even if contract addresses are missing.

Table. Organized required-work structure for implementation review.

Main Files

Primary file / path	Review purpose
server/src/routes/status.routes.js	API route registration
server/src/controllers/status.controller.js	API controller logic
server/src/index.js	JavaScript module
server/package.json	Package scripts/dependencies

Expected Deliverable

- [] Backend exposes GET /api/status.
- [] Endpoint returns VaultX PoC public status.
- [] Endpoint works without blockchain configuration.
- [] Endpoint can be used for simple health/status checks.

SC-13 - Add VaultXTreasuryLedger Contract

Domain	Timebox	Module Ref.	Implementation Mode
SOLIDITY	1 Business Day	SC-13	Standalone / non-blocking

Technical Target

Add a standalone treasury ledger contract for recording VaultX treasury allocation categories and off-chain reference hashes.

Current PoC Gap

The PoC has token and presale logic, but there is no separate contract for recording treasury categories, allocation labels, or governance-facing accounting references.

Required Work

Work Area	Organized Required Work
Create new contract	- contracts/VaultXTreasuryLedger.sol

Work Area	Organized Required Work
Add allocation struct (1)	- struct AllocationRecord { - string category; - uint256 amount; - string referenceURI; - bytes32 referenceHash; - uint256 createdAt; - bool active;
Add allocation struct (2)	- }
Storage model	- AllocationRecord[] public allocationRecords;
Contract functions (1)	- createAllocationRecord(- string calldata category, - uint256 amount, - string calldata referenceURI, - bytes32 referenceHash -) - updateAllocationStatus(- uint256 recordId, - bool active -) - getAllocationRecord(uint256 recordId) - getAllocationRecordCount()
Contract functions (2)	- only owner can create records - only owner can update record status - category cannot be empty - amount must be greater than 0 - recordId must exist
Events and observability	- event AllocationRecordCreated(uint256 indexed recordId, string category, uint256 amount, bytes32 referenceHash); - event AllocationRecordStatusUpdated(uint256 indexed recordId, bool active);

Table. Organized required-work structure for implementation review.

Main Files

Primary file / path	Review purpose
contracts/VaultXTreasuryLedger.sol	Smart contract source
Truffle/migrations/4_deploy_treasury_ledger.js	Deployment migration

Expected Deliverable

- [] Treasury ledger contract compiles.
- [] Owner can create allocation records.
- [] Owner can activate/deactivate records.
- [] Contract works independently from presale and asset registry.
- [] No frontend or backend change is required for this task.

3.0 Task Group 2 - 3 Business Days

Delivery focus: Standalone three-day improvements: compliance center page, document metadata API, and document anchor contract.

FE-21 - Build VaultX Compliance Center Page

Domain	Timebox	Module Ref.	Implementation Mode
FRONTEND	3 Business Days	FE-21	Standalone / non-blocking

Technical Target

Create a new standalone Compliance Center page that explains VaultX investor access, jurisdiction readiness, disclosure status, and risk review workflow using frontend-only content.

Current PoC Gap

The current frontend has presale, gallery, contact, and asset pages, but no dedicated compliance/disclosure section. For an RWA PoC, the product needs a professional compliance-facing page even before full backend or on-chain allowlist logic exists.

Required Work

Work Area	Organized Required Work
Add route	- /compliance
Create page	- src/containers/compliance/index.jsx
Create components	- src/components/compliance/ComplianceHero.jsx - src/components/compliance/InvestorAccessSteps.jsx - src/components/compliance/JurisdictionNoticePanel.jsx - src/components/compliance/RiskDisclosurePanel.jsx - src/components/compliance/DocumentReadinessPanel.jsx
Panel actions / messaging	- title - short VaultX compliance positioning text - public VaultX address - compliance status label
Panel actions / messaging	- Connect wallet - Request access - Review disclosures - Wait for approval - Participate when eligible - JurisdictionNoticePanel must show static placeholder notices. - RiskDisclosurePanel must show static RWA risk categories.
Panel actions / messaging (1)	- Asset memo placeholder - Legal wrapper placeholder - Token terms placeholder - Risk disclosure placeholder - Add navigation link in header or footer. - Do not call backend. - Do not call Solidity.
Panel actions / messaging (2)	- Use static frontend content only.

Table. Organized required-work structure for implementation review.

Main Files

Primary file / path	Review purpose
src/App.jsx	React page/component surface
src/containers/compliance/index.jsx	React page/component surface
src/components/compliance/ComplianceHero.jsx	React page/component surface
src/components/compliance/InvestorAccessSteps.jsx	React page/component surface
src/components/compliance/JurisdictionNoticePanel.jsx	React page/component surface
src/components/compliance/RiskDisclosurePanel.jsx	React page/component surface
src/components/compliance/DocumentReadinessPanel.jsx	React page/component surface
src/components/layout/Header/MainNavigation.jsx	React page/component surface
src/components/layout/Footer.jsx	React page/component surface

Expected Deliverable

- [] /compliance page works locally.
- [] Page matches VaultX design style.
- [] RWA compliance workflow is presented clearly.
- [] No backend dependency exists.
- [] No Solidity dependency exists.

BE-22 - Add Asset Document Metadata API

Domain	Timebox	Module Ref.	Implementation Mode
EXPRESS	3 Business Days	BE-22	Standalone / non-blocking

Technical Target

Add standalone backend endpoints for asset document metadata so the PoC can represent RWA document readiness without storing or serving real documents.

Current PoC Gap

The frontend can show asset cards, but the backend does not yet expose document metadata such as asset memo, risk disclosure, legal wrapper, valuation summary, or operating agreement placeholders.

Required Work

Work Area	Organized Required Work
Add routes	- GET /api/documents - GET /api/documents/:assetId
Create document data file	- server/src/data/documents.js
Each document metadata object must include (1)	<pre>- { - "assetId": "VX-MIA-001", - "documents": [- { - "id": "DOC-001", - "title": "Asset Summary Memo", - "type": "PDF",</pre>
Each document metadata object must include (2)	<pre>- "category": "asset_memo", - "status": "placeholder", - "url": null, - "updatedAt": "ISO_DATE_STRING" - } - }</pre>
Supported document categories	- asset_memo - legal_wrapper - valuation_summary - risk_disclosure - token_terms - GET /api/documents must return all document metadata. - GET /api/documents/:assetId must return metadata for one asset.
If assetId has no document record, return (1)	<pre>- { - "success": false, - "error": { - "code": "DOCUMENTS_NOT_FOUND", - "message": "No document metadata found for this asset" - } - }</pre>
If assetId has no document record, return (2)	- This API must not call blockchain RPC. - This API must not depend on presale routes. - This API must not depend on frontend state.

Table. Organized required-work structure for implementation review.

Main Files

Primary file / path	Review purpose
server/src/routes/documents.routes.js	API route registration
server/src/controllers/documents.controller.js	API controller logic
server/src/data/documents.js	Static PoC data source
server/src/index.js	JavaScript module

Expected Deliverable

- [] Backend exposes document metadata endpoints.
- [] Asset document readiness can be represented in the PoC.
- [] No real file upload is required.
- [] API is independent from frontend and Solidity work.

SC-23 - Add VaultXDocumentAnchor Contract

Domain	Timebox	Module Ref.	Implementation Mode
SOLIDITY	3 Business Days	SC-23	Standalone / non-blocking

Technical Target

Add a standalone document anchor contract for recording document hashes and metadata references for VaultX RWA assets.

Current PoC Gap

The PoC can display assets and contract data, but there is no on-chain way to anchor asset memo hashes, disclosure hashes, or legal document references.

Required Work

Work Area	Organized Required Work
Create contract	- contracts/VaultXDocumentAnchor.sol
Add document struct (1)	- struct DocumentRecord { - string assetId; - string documentType; - string metadataURI; - bytes32 documentHash; - uint256 createdAt; - bool active;
Add document struct (2)	- }
Storage model	- mapping(bytes32 => DocumentRecord) public documents; - mapping(bytes32 => bool) public documentExists;
Generate document key	- bytes32 documentKey = keccak256(- abi.encodePacked(assetId, documentType, documentHash) -);
Contract functions (1)	- anchorDocument(- string calldata assetId, - string calldata documentType, - string calldata metadataURI, - bytes32 documentHash -) - deactivateDocument(- string calldata assetId, - string calldata documentType, - bytes32 documentHash -) - getDocument(- string calldata assetId, - string calldata documentType, - bytes32 documentHash -)
Contract functions (2)	- bytes32 documentHash -)
Contract functions (3)	- only owner can anchor documents - only owner can deactivate documents - assetId cannot be empty - documentType cannot be empty - metadataURI cannot be empty - documentHash cannot be zero - duplicate document key is rejected
Events and observability	- event DocumentAnchored(string indexed assetId, string documentType, bytes32 indexed documentHash, string metadataURI); - event DocumentDeactivated(string indexed assetId, string documentType, bytes32 indexed documentHash);

Table. Organized required-work structure for implementation review.

Main Files

Primary file / path	Review purpose
contracts/VaultXDocumentAnchor.sol	Smart contract source
Truffle/migrations/5_deploy_document_anchor.js	Deployment migration

Expected Deliverable

- [] Document anchor contract compiles.
- [] Owner can anchor document metadata hashes.
- [] Duplicate document anchors are rejected.
- [] Owner can deactivate document anchors.
- [] Contract is independent from presale and frontend.

A.1 Senior Review Checklist

Area	Review Gate	Sign-off
Frontend	Asset detail drawer and Compliance Center page match VaultX styling, use clean inactive states, and do not require backend/Solidity dependencies.	☐ Approved
Express API	Status and document metadata endpoints return stable success/data and error/code/message response shapes without RPC dependency.	☐ Approved
Solidity	Treasury ledger and document anchor contracts compile, validate inputs, emit events, and remain independent from existing presale logic.	☐ Approved
Spacing/Layout	PDF pages have no clipped text, oversized blank gaps, or messy Required Work sections.	☐ Approved
Local QA	Routes, endpoints, and contracts can be reviewed locally by the assigned developers.	☐ Approved

Table 3. Senior review gates.

A.2 Approval Record

Name	Role / Title	Signature	Date

Prepared for internal VaultX PoC engineering review. This document is not a production audit report and should not replace contract security review, legal review, or deployment approval.